

Bienvenue :-)

INFO2 - Programmation Web Avancée

Informations utiles

-  Téléphone : en silencieux SVP
-  Notes : prenez-en, tout n'est pas écrit sur le support de cours (l'oral n'est pas encore mort : exemples, détails, cas réels ...)
-  Emargement : imposé CM/TP, comme dans les autres matières
-  TP : avec vos PC aussi bien que ceux de l'université
-  IA : autorisé ou interdite en fonction du TP (cf. sujet)



Programmation Web Avancée

Sup Galilée – INFO2
Promotion 2028

Benoît VILLA

Ingénieur R&D – Ilex International



benoit.villa@univ-paris13.fr

Présentation du cours

Objectifs et positionnement

Objectifs :

- Acquérir des notions avancées en programmation Web
- Manipulation de documents Web
- Exploitation de documents XML / JSON
- Compréhension et mise en place d'un service Web (client et serveur)
- Interfaçage avec des bases de données relationnelle
- Utilisation de bibliothèques pour le développement de client/serveur Web

INFO1

- Réseaux
- Programmation web
- Bases de données

INFO2

- Prog. web avancée

INFO3

- Cryptographie
- Cloud Computing
- Sécurité informatique

Programme des enseignements

Différentes notions liées au Web (HTTP, HTML, modèles client/serveur, ...)

Présentation de la structure d'un service Web

Tomcat, Servlets et JSP

Formats de données du web : XML et JSON

Services web: REST, SOAP, GraphQL; JAX-RS et JAX-WS

Outils : Java, Tomcat, Maven, Eclipse, ...

Accès à une base de données SQL, ORM (Hibernate)

Répartition des enseignements

Séances le mardi après-midi

- 1h30 de cours
- 3h de TP

Contrôle des connaissances :

- QCM
- Devoir (préparé en partie pendant les TPs)

Internet

Réseau de réseaux, basé sur architecture IP (TCP et UDP **cours Réseau*)

Définit l'infrastructure et les protocoles de communication

Modèles client/serveur ou pair à pair

Une multitude de services :

- téléphonie, courrier électronique, visio, ...
- transfert et partage de fichiers
- vidéos (streams, VOD, TV), musique, radio, ...
- connexion à distance
- pages web (www)
- applications

World Wide Web (www)

Architecture pour accéder à des documents/ressources numériques de tous types, liés entre eux et situés sur des machines reliées par Internet.

Créé par Tim Berners-Lee, début 1990

3 notions essentielles :

- URL (Uniform Resource Locator) : localisation/adresse du document
- HTTP/HTTPS : protocole de communication (transfert)
- HTML : langage de structuration -à l'aide de balises- et affichage du contenu

HyperText Markup Language (HTML)

Langage de structuration du contenu, notamment texte, à l'aide de balises (définition logique d'un document)

Différents types d'objets : textes, tableaux, images, sons, vidéos, etc.

Ensemble de commandes de formatage, délimitant le début et la fin d'un environnement (balises)

HTML : norme, syntaxe et validation

HTML : langage utilisé pour décrire et structurer le contenu d'une page web.

Norme évolutive, syntaxe souple.

Devrait être respectée par tout document HTML

En pratique :

- certains documents non valides (norme non respectée)
- navigateurs tolérants aux erreurs de syntaxe
- standards en constante évolution

(WHATWG – HTML Living standard, cf. <https://html.spec.whatwg.org/multipage/>)

Vers le web dynamique et interactif

HTML seul : pas/peu d'interactivité avec l'utilisateur. Les pages sont "statiques"

Pages dynamiques : dépendantes des manipulations l'utilisateur

Génération dynamique en fonction de paramètres de l'utilisateur

Principaux types d'interactions :

a) côté client

b) côté serveur

→ On parle de programmation web

Programmation Web côté client

Exécution réalisée sur le client

Navigateur capable de réaliser l'exécution

Transfert du code, depuis le serveur vers le client :

- embarqué dans la page HTML (tout en un)
- via un fichier dédié (à privilégier)

Exemple : JavaScript

Programmation Web côté serveur

Exécution code coté serveur :

- PHP
- Java (Servlet / JSP)
- C# / ASP .Net
- Ruby
- Python
- ...

Résultat de l'exécution : la page HTML générée est envoyée par le serveur au navigateur

Programmation Web côté client ou serveur ?

Le mieux est d'utiliser les deux :

1. Côté client

- Améliore l'ergonomie pour l'utilisateur (ex: affichage calendrier pour choisir une date, détection erreur de saisie, ...)
- Rend le service interactif, ergonomique et agréable

2. Côté serveur

- Permet d'effectuer la majorité des traitements
- Contrôle les données envoyées, soyez (presque ;-)) paranoïaques !

(ne **JAMAIS** faire confiance aux données ni aux contrôles fait côté client !)



Réseau, modèle OSI

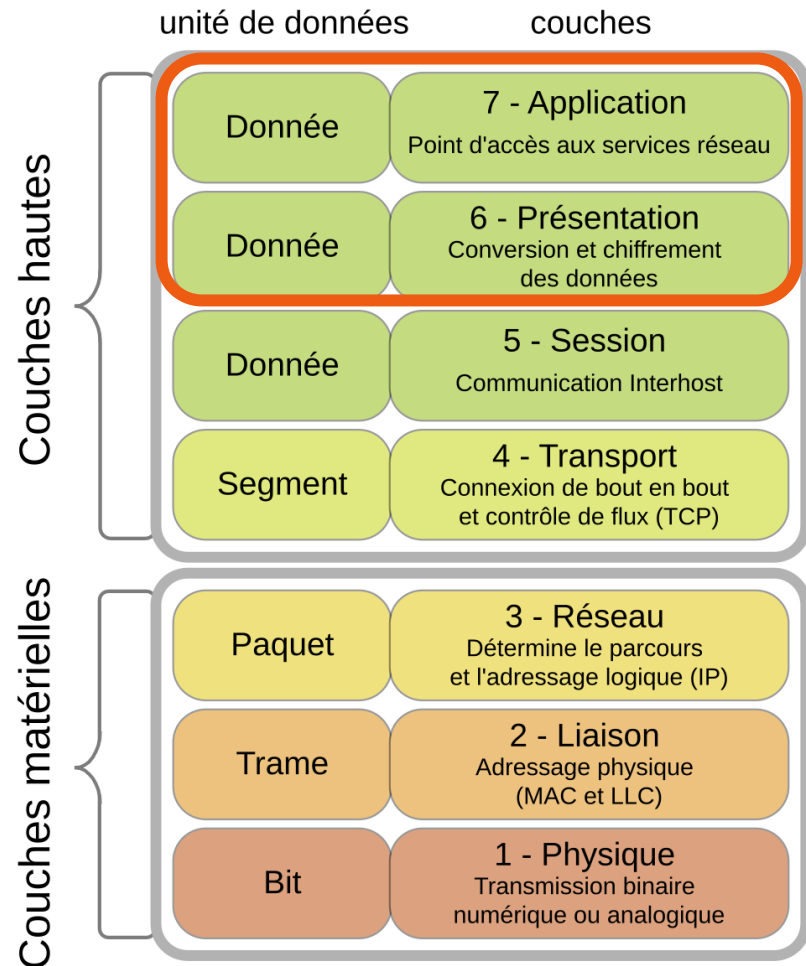
Selon le modèle OSI :

HTTP :

- couche 7

HTTPS = HTTP over TLS

- TLS (chiffrement): couche 6
- HTTP : couche 7



Hyper Text Transfer Protocol (HTTP)

Protocole définissant le dialogue entre un client et un serveur web
Ports standard d'écoute du serveur : 80 (HTTP) / 443 (HTTPS)

Contenu d'une transaction :

- le type de message : requête ou réponse (commande HTTP)
- entêtes techniques
- une ligne vide
- contenu (parfois vide)

💡 Pour aller plus loin ... :

- <https://developer.mozilla.org/fr/docs/Web/HTTP>
- <https://developer.mozilla.org/fr/docs/Web/HTTP/Guides/Overview>

Exemple de transaction

Récupération d'une page web (/index.html)

Demande de la page /index.html

GET / HTTP/2

Envoi des informations d'entête (configuration du client, documents acceptés)

Host: www.univ-spn.fr

UserAgent: Mozilla/5.0 (Windows NT 10.0; win64; x64; rv:155.0)

Gecko/20100101 Firefox/155.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9;q=0.8

Accept-Language: fr,fr-FR;q=0.8,en-US;q=0.5,en;q=0.3

Accept-Encoding: gzip, deflate, br

Envoi d'une ligne vide (fin de l'entête) et un contenu vide

Exemple de transaction

Récupération d'une page web (/index.html)

Réponse du serveur sous d'un code indiquant l'état de la requête
HTTP/2 200 OK

Envoi des informations sur le serveur (configuration) et du document demandé

Date: Tue, 02 Sep 2026 08:50:28 GMT

Server: Apache/2.4.41 (Ubuntu)

Last-Modified: Mon, 01 Sep 2026 15:02:06 GMT

Accept-Ranges: bytes

Content-Length: 79991

Expires: Tue, 15 Sep 2026 07:04:03 GMT

Content-Type: text/html; charset=UTF-8

Content-Encoding: gzip

[...]

Envoi d'une ligne vide (fin de l'entête) et un contenu vide si la réalisation de la requête est un succès

Récap - format des requêtes et réponses

Format des requêtes

type de la requête : METHOD URL version de HTTP
en-tête

une ligne vide

un contenu éventuel

METHOD : GET, POST, HEAD

Format des réponses

code de réponse : version HTTP code description
entête

une ligne vide

contenu de la réponse

Quelques code réponses HTTP

1xx : information

100 : Continue (serveur prêt pour recevoir la suite de la requête)

2xx : succès

200 : OK

3xx : redirection

301 : Redirection

4xx : erreur liée au client

401 : Unauthorized (authentification requise)

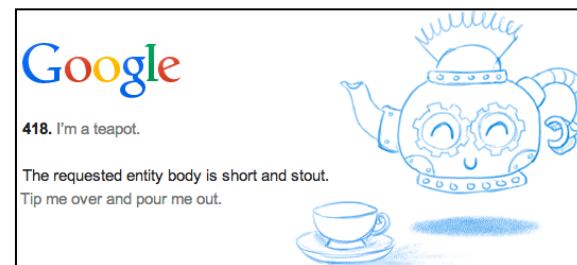
403 : Forbidden

404 : Not found (tiens tiens, un lien mort ?)

418 : I'm a teapot (RFC 2324, 1^{er} avril 1998)

5xx Erreur liée au serveur

500 : Internal Server Error (erreur de config. ou de programmation)



🔧 Liste détaillée : <https://developer.mozilla.org/fr/docs/Web/HTTP/Reference/Status>

HTTP sans état / stateless

HTTP est un protocole « sans état », permettant une gestion légère des transactions.

(requête traitée, réponse envoyée puis échanges oubliés du serveur)

Pour conserver des informations entre plusieurs transactions, utilisation de ...

- sessions
- cookies
- champs cachés dans un formulaire

Le web d'aujourd'hui : riche mais complexe

Architecture distribuée

- Navigateurs ↔ CDN ↔ serveurs applicatifs ↔ API ↔ bases de données
- HTTP(S), DNS, TLS, WebSockets, HTTP/2–3, QUIC

Applications riches : JS/TS, Web APIs, frameworks

- SPA, SSR, SSG, WebAssembly, PWA
- Interfaces temps réel et multimédia

Cloud & edge (**cf. cours INFO3)

Le web d'aujourd'hui : riche mais complexe



Web et IA

- Recherche et recommandation personnalisées
- Agents et assistants connectés au Web
- Génération de contenu, traitements grande échelle



Du web mais pas n'importe comment (**cf. Sécu info INFO3)

Sécurité : e-identité, attaques XSS, CSRF, supply chain, API

Vie privée, souveraineté et réglementation

Accessibilité, performance et sobriété numérique

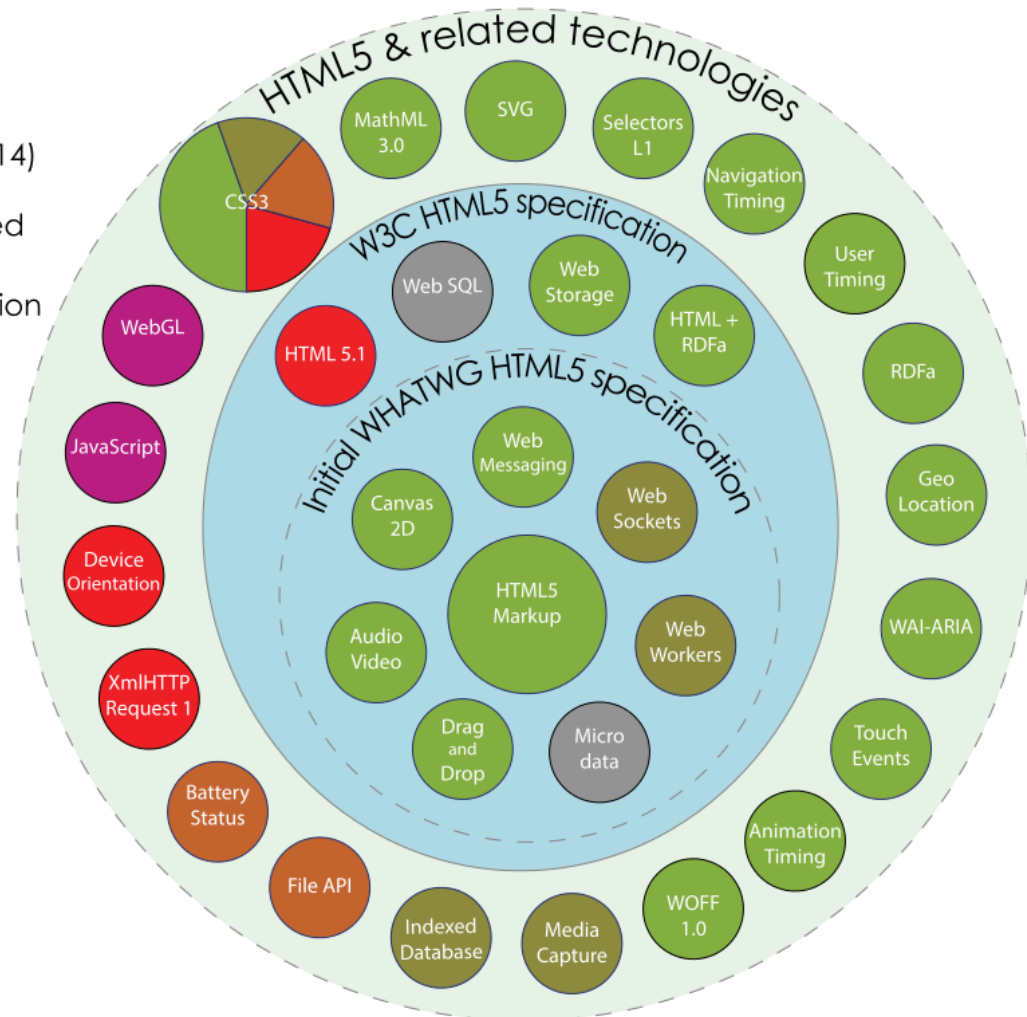


le Web n'est plus juste un ensemble de pages hypertexte : c'est une **plateforme logicielle distribuée mondiale**, sur laquelle s'exécutent des applications complexes.

HTML5

Taxonomy & Status (October 2014)

-  Recommendation/Proposed
-  Candidate Recommendation
-  Last Call
-  Working Draft
-  Non-W3C Specifications
-  Deprecated or inactive



Source Wikipedia, article HTML 5

By Mercury999 (Own work) [CC-BY-SA-4.0 (<http://creativecommons.org/licenses/by-sa/4.0>)], via Wikimedia Commons

En perpétuelle évolution

- Besoins et usages : en permanente évolution, par ex. explosion de l'utilisation du télétravail et de l'audio-visioconférence
- Appareils : des PC de bureau aux appareils mobiles
- Standards : HTML / CSS / Javascript en évolution régulière ...
- Navigateurs : innovations et mise à jour régulières



Programmation Web Avancée

JSP et Servlet

Benoît VILLA

Ingénieur R&D

benoit.villa@univ-paris13.fr

Introduction

Pages HTML dynamiques créées lors de la requête

Code à exécuter inclus dans la page HTML

Gestion de la partie dynamique coté serveur : server side include

Principe général (coté serveur Web) :

- Réception d'une requête
- Passage au programme qui traite la partie de la page le concernant
- Génération de la partie dynamique de la page par le programme
- Renvoi de l'ensemble de la page HTML créée au client Web

Servlet et JSP

Servlet : Code Java contenant des données HTML

Partie dynamique de la page Web

Rôle :

- Aiguillage des données
- Accès aux autres méthodes que celles de service
- Définition d'autres méthodes

JSP : Page HTML contenant du code Java

Partie statique de la page Web

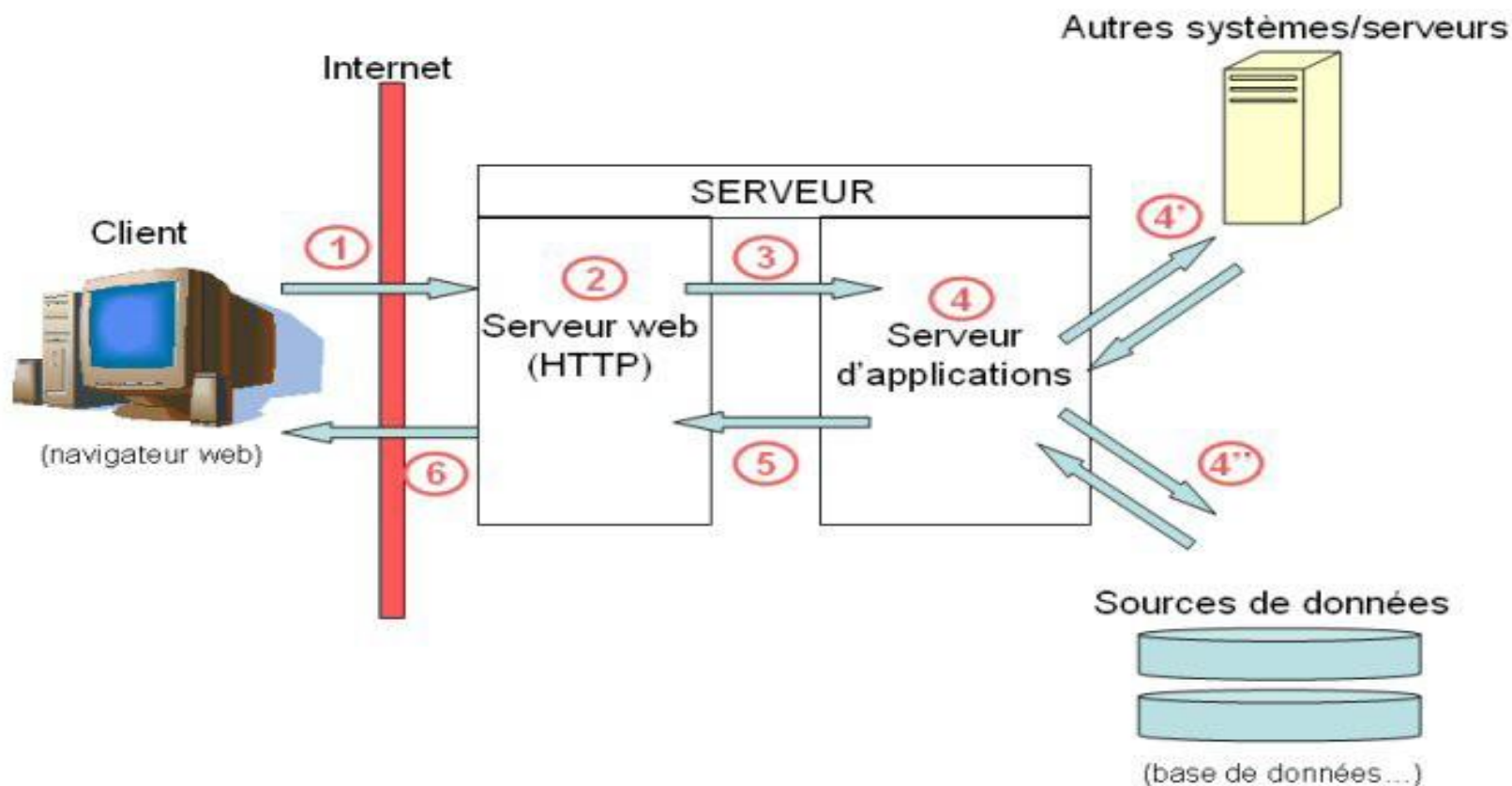
Rôle :

- Inclusions d'autres JSP / intégration de composants
- Gère la partie affichage

Globalement, répartition en fonction du volume de code Java et HTML, et de la couche dans laquelle on se trouve

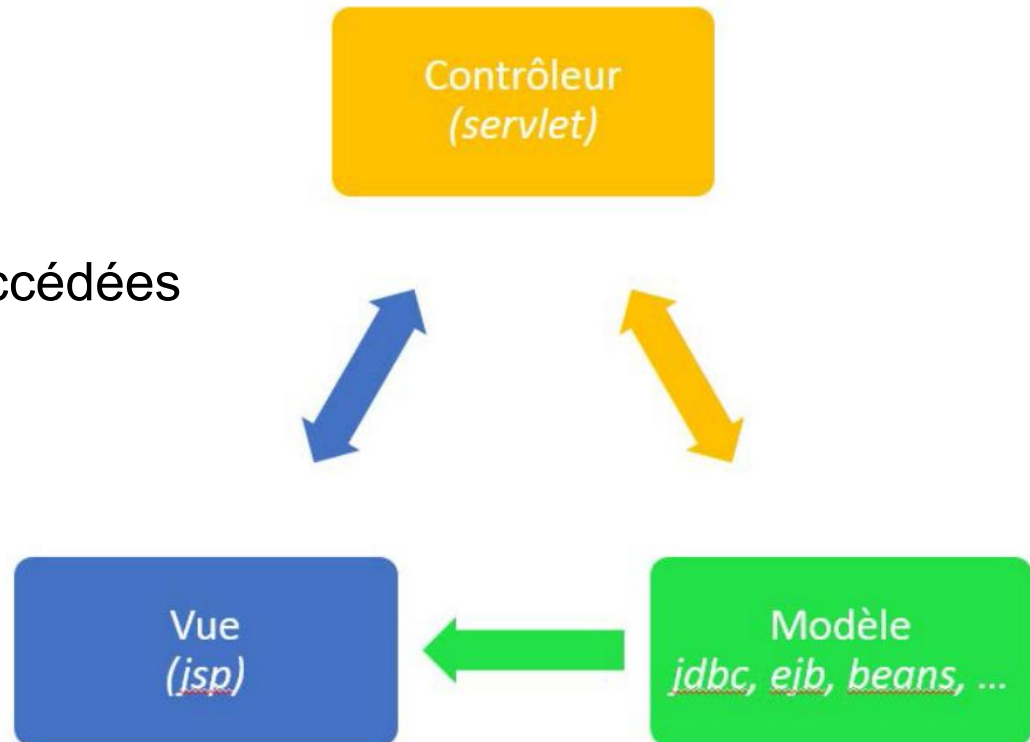
JSP et Servlet

Architecture globale (n-tiers)



Architecture MVC

- Modèle : données accédées par le code Java
- Vue : JSP
- Contrôleur : servlets



Architecture MVC

Echanges Servlet/JSP

Lancement de la JSP dans la servlet :

```
public void doPost (HttpServletRequest request , HttpServletResponse response) {  
    ServletContext context = getServletContext(); // heritee de GenericServlet  
    RequestDispatcher dispatcher = context.getRequestDispatcher("/maPage1.jsp");  
    dispatcher.forward(request, response);  
}
```

Passage de valeurs à la JSP par la servlet (setAttribute()) :

```
public void doPost (HttpServletRequest request, HttpServletResponse response) {  
    // appel des methodes sur les objets  
    ArrayList theList<String> = ... // un objet à passer  
  
    // ajout à la requête  
    request.setAttribute("nomDelObjet", theList);  
  
    ServletContext context = getServletContext();  
    context.getRequestDispatcher("/jspAAppeler.jsp").forward(request, response);  
}
```

Architecture MVC

Echanges Servlet/JSP

Extraction par la JSP des objets de la requête (getAttribute()) :

```
<% ArrayList<String> theList =  
    (ArrayList) request.getAttribute("nomDelObjet");  
    for (String item : theList) {  
        //Utilisation de theList pour affichage  
    }  
%>
```

Exemple de JSP

date.jsp :

```
<html><head><title>Obtenu par une JSP</title></head>
<body>
<h3>Exemple de JSP</h3><hr>
Date du jour : <%= new java.util.Date()%>
</body>
</html>
```

Traitement de date.jsp lorsque le client demande l'URL

<http://serveurWeb:<port>/<chemin>/date.jsp>

Ex: <http://127.0.0.1:8080/tp1/date.jsp>

Moteurs JSP et servlet

Moteur : nécessaire à l'exécution des JSP et Servlets.

- Tomcat (<http://tomcat.apache.org/>)
- Jetty (<http://www.mortbay.org/jetty/>)
- IBM WebSphere (ibm.com/software/websphere/)
- WebLogic
- GlassFish

Par convention, les ports TCP suivants sont utilisés :

- HTTP : 8080
- HTTPS : 8443
- Remarque: n° de port +8000 par rapport à un serveur web habituel

Tomcat

Projet de la fondation Apache

Référence en matière de moteurs de servlets

Disponible sur plateformes UNIX et Windows

Composants :

- Serveur web

- Moteur de servlets : Catalina

- Compilateur de JSP : Jasper

Exemples de JSP :

.../webapps/examples/jsp/index.html

Exécution d'une JSP

1. Construction par le moteur de JSP d'une servlet (traduction de la JSP)
2. Compilation et exécution de la servlet

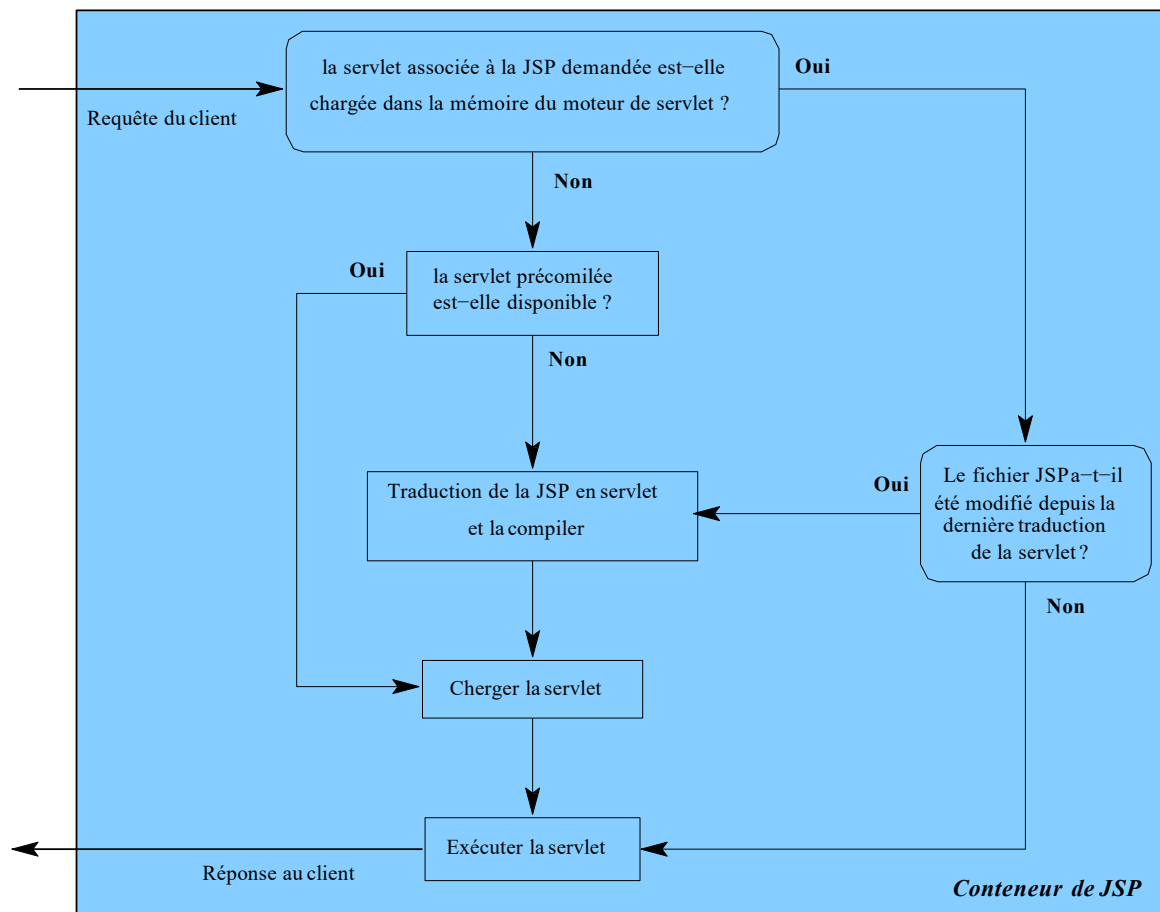
Actions automatiques, vous n'avez pas besoin d'intervenir !

Exécution d'une JSP

```
package jsp;

...
public class _0002fjsp_0002fjsp_0002fdate_0002ejspdate.jsp_1 extends
HttpJspBase {
    ...
    public void jspService( HttpServletRequest request , HttpServletResponse respo
nse) throws IOException , ServletException {
        ...
        _jspx_init ();
        ...
        pageContext = jspFactory.getPageContext ( ... ); s
ession = pageContext.getSession ();
        out = pageContext.getOut ();
        // HTML
        // begin [ file = "/.../ examples/ jsp/ date .jsp "; from =( 0,0); to =( 4,24)]
        out.write("<html><head><title>Obtenu par une JSP</title></head>¥r ¥n
<body>¥r ¥n¥r ¥n<h3>Bonjour de ma part</h3> < hr>¥r ¥n
        La date courante est : " );
        // end
        // begin [ file = "/.../ examples/ jsp/ date .jsp "; from =( 4,27) to =( 4,49)]
        out.print( new java.util.Date() );
        // end
        // HTML
        // begin [ file = "/.../ examples/ jsp/ date .jsp "; from =( 4,51); to =( 6,7)]
        out.write("¥r ¥n</body>¥r ¥n</html>"); // end
        ...
    }
}
```

Processus d'exécution de la JSP



Description d'une servlet

Définition d'une servlet : implémentation Java d'un mécanisme de requête/réponse, conçue indépendamment des protocoles

Possibilité d'encapsulation de données dans :

- Des objets génériques (*request*, *response*, *context*)
- Des objets spécifiques au protocole HTTP (méthode, type MMIE, headers, session, cookies, ...)

API servlet

package jakarta.servlet.http.HttpServlet (**javax.servlet = trop ancien**)

HttpServlet propose une méthode (de classe) par méthode HTTP, nécessitant la surcharge d'une de ces méthodes pour définir le code à exécuter :

protected void **doGet**(HttpServletRequest req, HttpServletResponse resp)
throws ServletException, IOException

protected void **doPost** (HttpServletRequest req, HttpServletResponse resp)
throws ServletException, IOException

Autres méthodes : doDelete, doHead, doOptions, doPut, doTrace, init, destroy, ...

API servlet

Accès à travers les objets requête et réponse passés en paramètres des méthodes de service :

ServletRequest : `getParameter`

ServletResponse : `getWriter`

HttpServletRequest : `getCookies`, `getHeader`,
`getMethod`, `getSession`

HttpServletResponse : `addCookie`, `addHeader`,
`sendError`, `sendRedirect`

Exemple de code de servlet

protocole HTTP

```
import javax.servlet.*;
import javax.servlet.http.*;

public class NewServlet extends HttpServlet {
    public void init(ServletConfig config) throws ServletException {
        super.init(config); ... }
    public void destroy() { ... }

    protected void doGet(HttpServletRequest request , HttpServletResponse response)
        throws ServletException , IOException {
        response.setContentType(" text/ html" );
        PrintWriter out = response.getWriter()
        ;
        out.println("<html><head><title>Hello page</title></head>");

        out.println("<body><h1>Hello " + request.getParameter("name") +
        </h1></body></html>");
    }

    protected void doPost( HttpServletRequest request , HttpServletResponse response) thro
    ws ServletException , IOException { ... }
}
```

Réflexion sur l'utilisation des servlet

Avantages : composants

Simple : Classes Java

Pratique : Code minimum (uniquement gestion du cycle de vie et du traitement de la requête – prise en charge des autres aspects par le conteneur)

Sûr : isolation du serveur par le conteneur.

Inconvénients :

Production de la réponse peu pratique : Beaucoup de `out.println( )`

Difficulté de compréhension du code HTML généré

Description d'une JSP

Principe : Rédaction d'une page Web dynamique comme si elle était statique, en incluant que le code nécessaire

Fonctionnalités identiques à HttpServlet :

Mécanisme de requête/réponse avec accès aux mêmes données et objets qu'une servlet, et possibilité d'inclusion ou de redirection de la requête vers une autre JSP ou servlet

Spécificité du protocole HTTP :

Génération de différents types de contenus : HTML, XML, SVG, etc.

Gestion des méthodes, headers, cookies, sessions, etc.

Description d'une JSP

Une JSP est composée de 3 parties :

- Scriptlets `<% ... %>`
- Déclarations `<%! ... %>`
- Expressions `<%= ... %>`

Scriptlets

Balises : `<% code %>`

```
<% response.setContentType("text/plain"); %>
```

Exemple :

```
<%  
    String[] langages = {"Java", "C++", "Smalltalk", "Simula 67"};  
    out.println("<h3>Principaux langages orientés objets: </h3>");  
    for (int i=0; i < langages.length; i++) {  
        out.println("<p>" + langages[i] + "</p>");  
    }  
%>
```

Déclarations

Balises : `<%! %>`

Code Java

Syntaxe script :

```
<%! private int nombreEtudiants = 23; %>
```

Exemple :

```
<%!  
int random4 ( ) {  
    return (int)(Math.random() * 4);  
}  
%>
```

Expression

Balises : `<%= code %>`

Expressions Java insérées dans le code de la page

Expressions évaluées, renvoyant un objet String ou un type primitif

Syntaxe script

```
<%= new java.util.date () %>
```

Equivalent à

```
<% out.println(new java.util.date ()); %>
```

Directives

Balises : `<%@ code %>`

Regroupement des informations globales relatives à la page 3
types :

page : modification des données de la page (importation de paquetages, gestion des sessions, etc.

```
<%@ page import="java.util.*" %>
```

include : inclusion de fichiers, de jsp ou de servlets

```
<%@ include page="/monJSP.jsp" flush="true" %>
```

taglib : utilisation de bibliothèques de balises personnalisées

```
<%@ taglib uri="..." prefix="..." %>
```

Objets prédéfinis dans un JSP

- out : flux de sortie
- request : objet correspondant à la requête (HttpServletRequest)
- response : objet correspondant à la réponse (HttpServletResponse)

Mais aussi : session, application, page

Exemple de JSP

comptabilisation du nombre d'appels de la page

```
<html>
  <head><title>Declarations et expressions</title></head>
  <body>
    <h1>Declarations JSP</h1>

    <%! private int accessCount = 0; %>

    <p>Cette page a ete accedee <%= ++accessCount %> fois depuis le demarrage du
    serveur </p>

  </body>
</html>
```

Bibliothèques de tags

Prise en charge de fonctionnalités sans écrire de code Java

Liaison à des classes Java, mappage des tags à l'exécution

Exemples (Java Standard Tag Library – JSTL) :

Core :

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

Internationalisation :

```
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
```

Fonctions :

```
<%@ taglib prefix="fn" uri="http://java.sun.com/jsp/jstl/functions" %>
```

Autres possibilités mais déconseillées : XML, SQL

Packaging et déploiement

Création d'un fichier .war (Web Archive)

Contenu du fichier :

Fichiers Web de l'application (HTML, JSP, js, css...)

Fichiers de configuration : Répertoire META-INF

- fichier MANIFEST.MF : informations sur l'archive

Contenu de l'application : Répertoire WEB-INF

- Répertoire *classes* : autres classes de l'application (beans, etc.)
- Répertoire *lib* : bibliothèques supplémentaires (jars) – optionnel
- Répertoire *src* : sources Java – optionnel

Packaging et déploiement

Objectif : Exécution d'une application par un conteneur Web

- Dépôt dans le répertoire dédié du serveur (webapps pour Tomcat)
- Lecture des fichiers .war au (re)démarrage du serveur :
 - Analyse du fichier .war et des paramètres de configuration du descripteur de déploiement
 - Création du répertoire correspondant dans le dossier webapps
 - Mappage des URL de l'application vers le répertoire créé

Conclusion

Utilisation des Servlet/JSP avec d'autres technologies Web : CSS, JavaScript, XML, AJAX, etc.

Importance de la réflexion sur la structure de l'application (architecture MVC notamment)

Utilisation de frameworks :

Aide à la mise en place de services complexes et assurance de rigueur dans le développement

- Exemple: Spring